



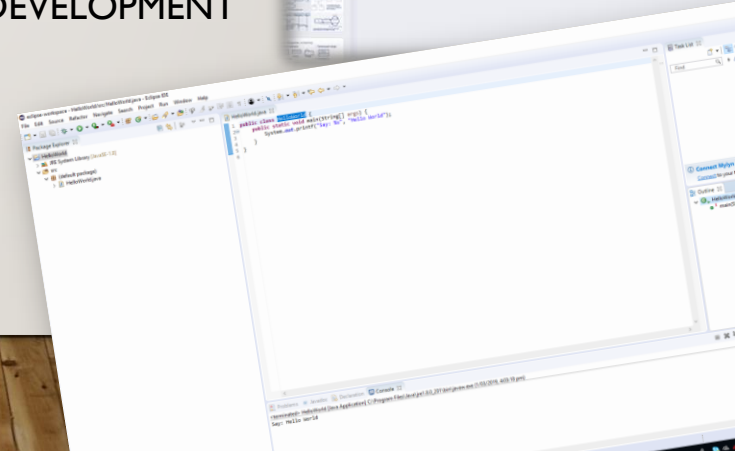
THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tamaki Makaurau
NEW ZEALAND

JAVA GUI – I

AWT FRAMEWORK AND EVENT HANDLING

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

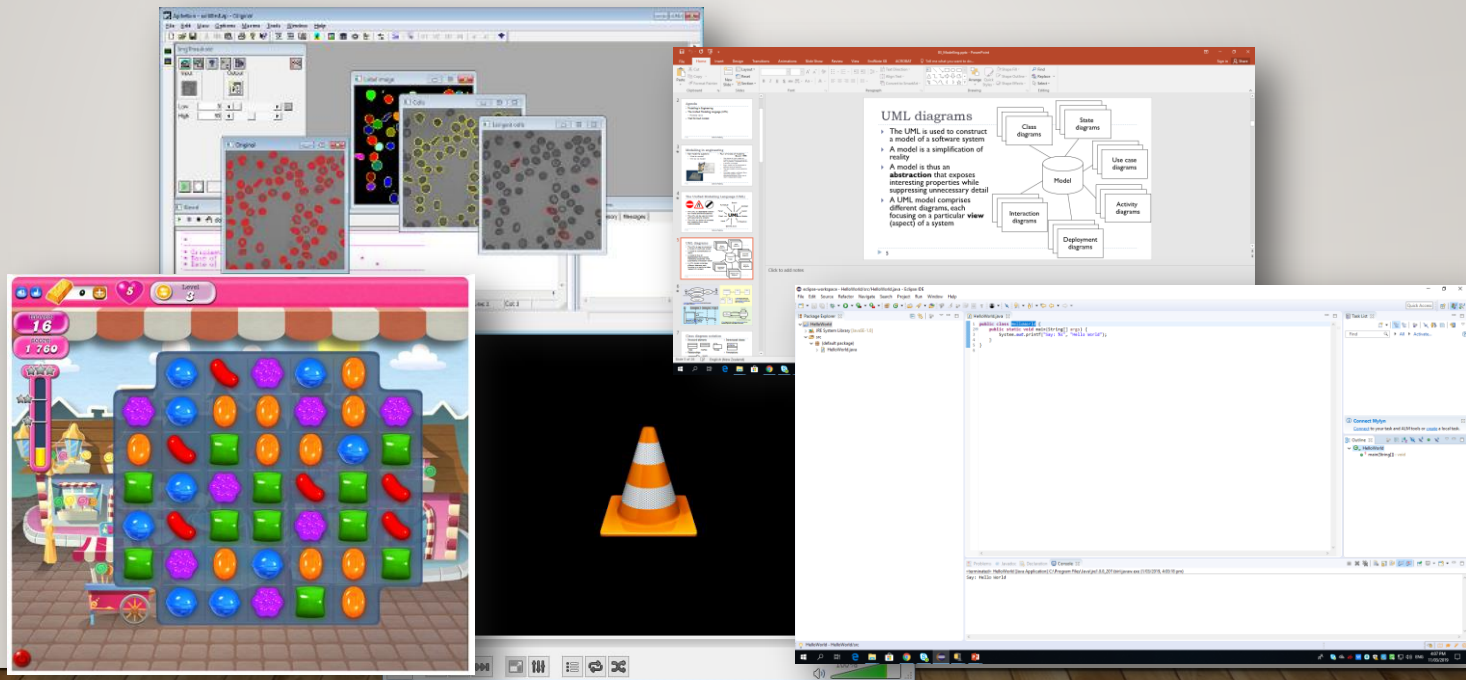
PARAMVIR SINGH



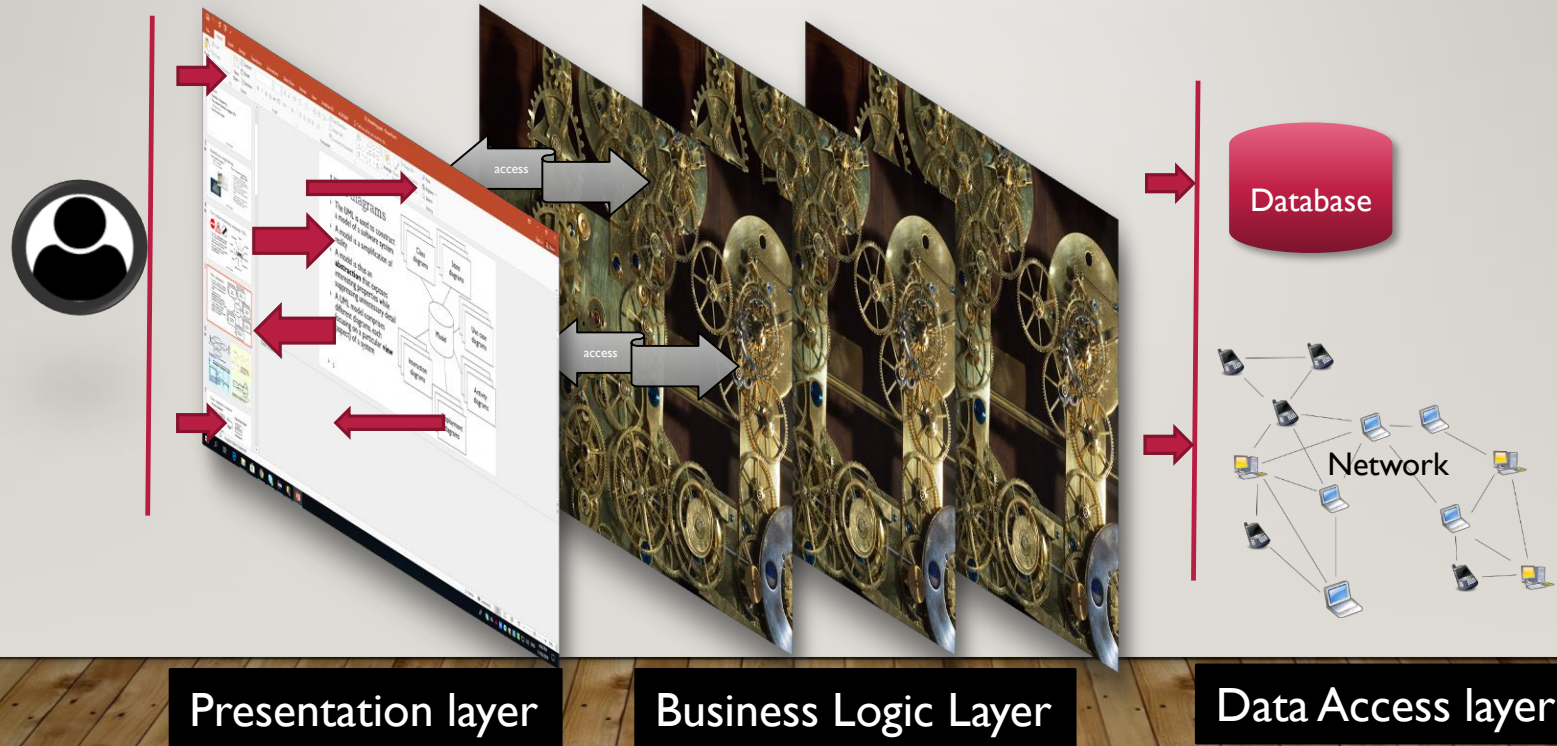
AGENDA

- GUI in 3-Layered Software Architecture
- Abstract Windowing Toolkit (AWT)
 - GUI component class hierarchy
 - Layout management
 - Event handling
 - Design choices for GUI programs
- Code resources
 - PalindromeGUI

GUI (GRAPHICAL USER INTERFACE)



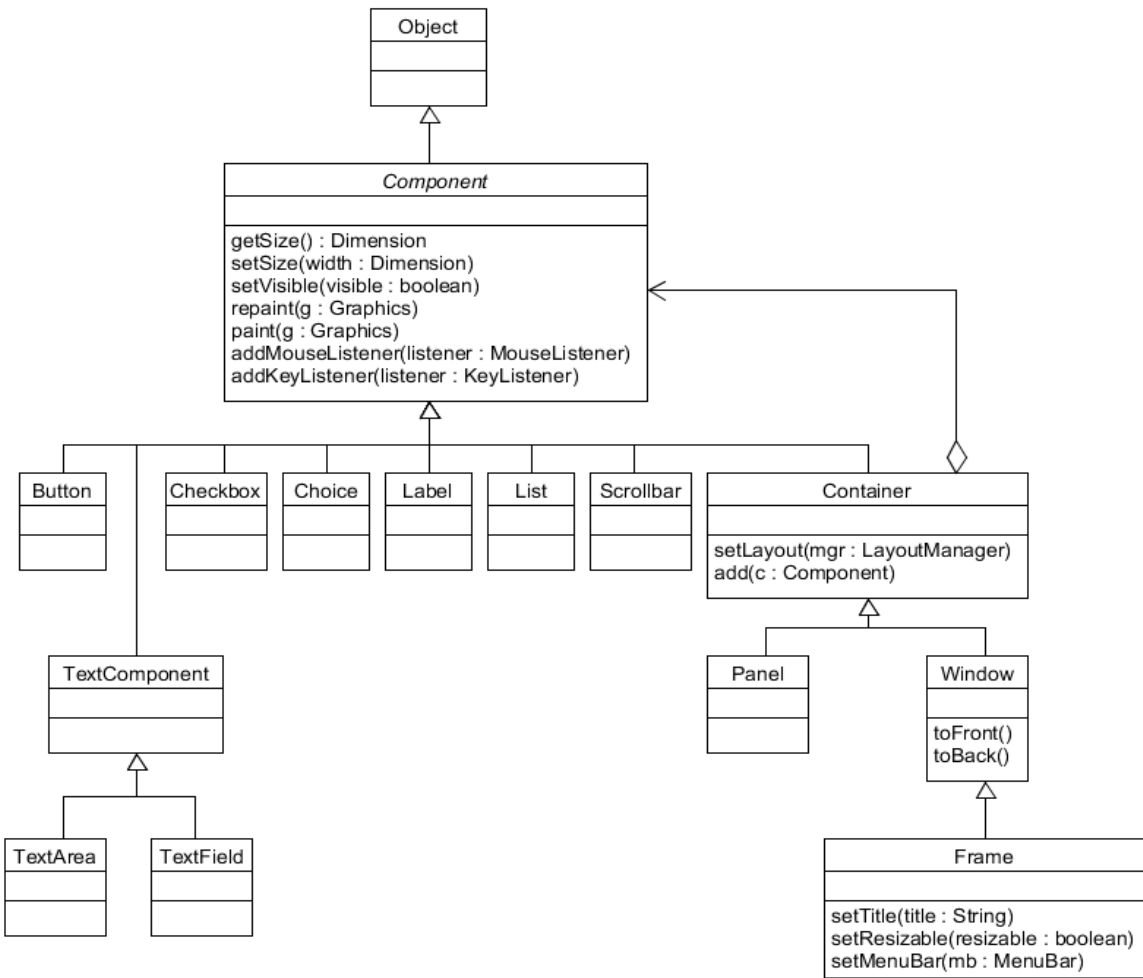
GUI (GRAPHICAL USER INTERFACE) IN *THREE-LAYERED SOFTWARE ARCHITECTURE*



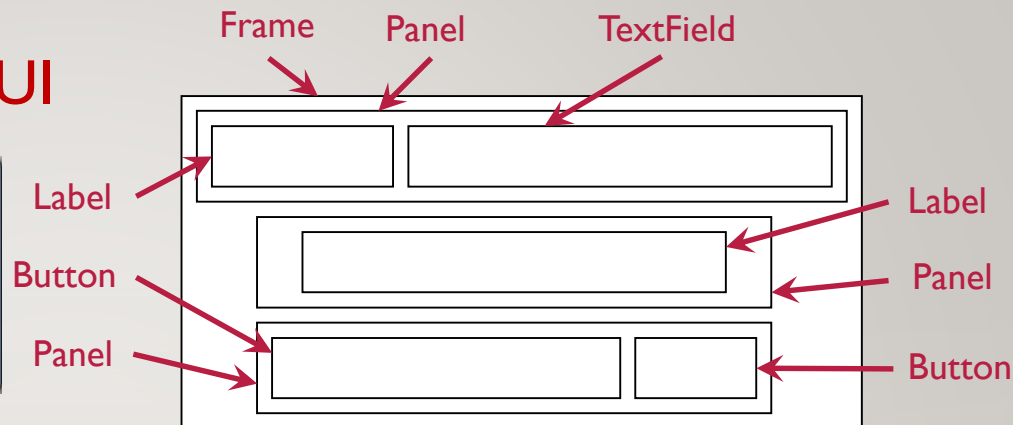
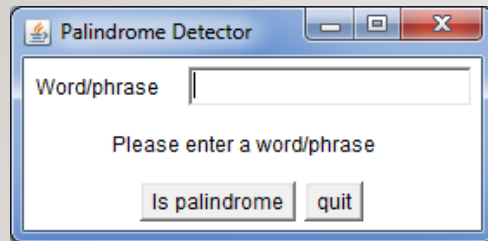
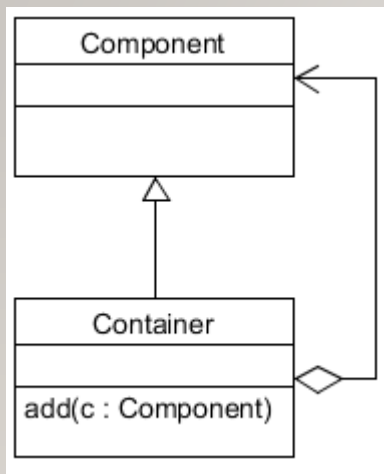
AWT (ABSTRACT WINDOWING TOOLKIT)

- Java's AWT is a simple framework for developing graphical user interfaces (GUIs)
- Key AWT packages include:
 - **java.awt**
 - Contains core GUI **component** classes
 - Components are also known as **widgets** and may be “simple” (e.g. buttons, textfields) or “composite” (e.g. windows, frames, panels)
 - Includes **layout managers** that layout *composite* components' children
 - **java.awt.event**
 - Contains classes representing GUI **events**
 - Events represent things like window closing and restoration, button presses, mouse movement, keystrokes, etc. Components generate events
 - Provides **listener interfaces**
 - Listener interfaces are provided that are intended to be implemented by application developers; listener objects can be notified of events

JAVA AWT CLASS HIERARCHY

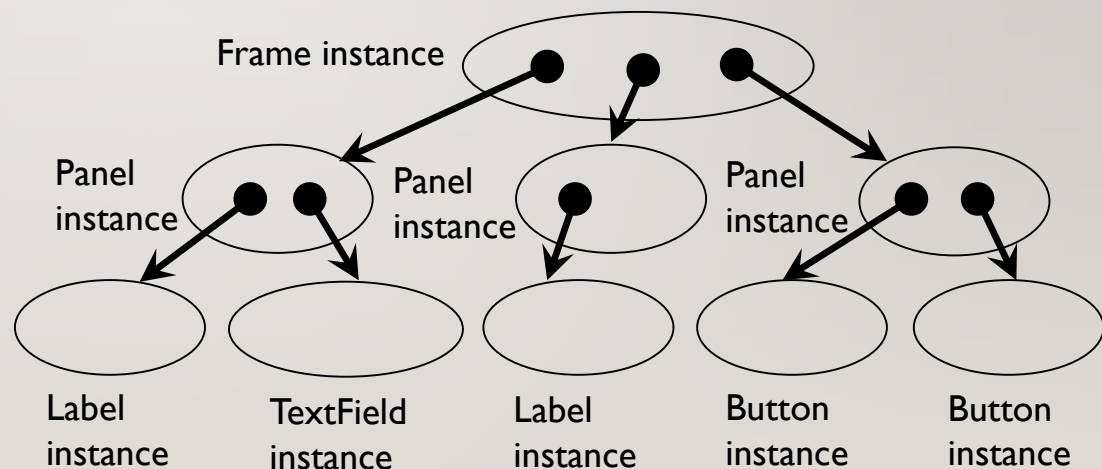


EXAMPLE APP: **PALINDROME**GUI



```

class Container extends Component {
    private List<Component> children;
    ...
    public void add( Component c ) {
        children.add( c );
    }
}
    
```



EXAMPLE APP: PALINDROME GUI

```
import java.awt.*;

public class PalindromeGUI extends Frame {

    public PalindromeGUI() {
        super( "Palindrome detector" );
        setUp();

        // Method inherited from class Window –
        // causes this object to be sized according
        // to its preferred size, based on its
        // children components.
        pack();

        setVisible( true );
    }

    public static void main( String[] args ) {
        PalindromeGUI palindromeGUI = new
        PalindromeGUI();
    }
}
```

```
private void setUp() {
    // Create intermediate panels.
    Panel inputPanel = new Panel();
    Panel outputPanel = new Panel();
    Panel buttonPanel = new Panel();

    // Create simple components.
    Label prompt = new Label( "Word/phrase" );
    TextField text = new TextField( 20 );
    Label output = new Label(
        "Please enter a word/phrase" );
    Button isPalindrome = new Button( "Is palindrome" );
    Button quit = new Button( "quit" );

    // Add components to intermediate panels.
    inputPanel.add( prompt );
    inputPanel.add( text );
    outputPanel.add( output );
    buttonPanel.add( isPalindrome );
    buttonPanel.add( quit );

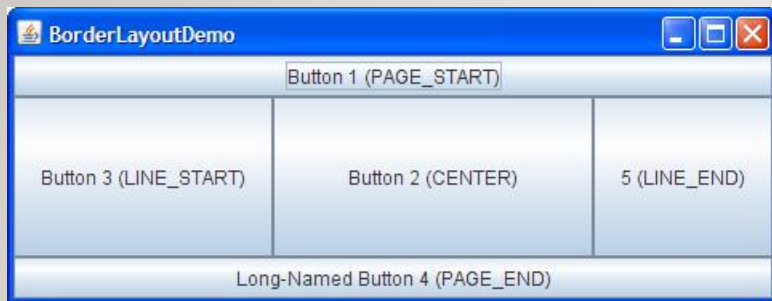
    // Add panels to this object.
    add( inputPanel );
    add( outputPanel );
    add( buttonPanel );
}
}
```


TRAVERSING THE NESTED STRUCTURE

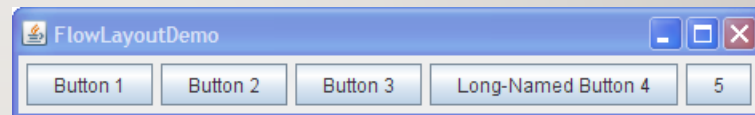
- To paint the GUI (to display it), method **paint()** is called on the root node (the **Frame**).
- This causes the **Frame** to visit its children and calling **paint()** on them; they in turn visit their children and so on

```
class Container extends Component {  
    private List<Component> children;  
    ...  
    public void add( Component c ) {  
        children.add( c );  
    }  
  
    public void paint( Graphics g ) {  
        for( Component c : children )  
            c.paint( );  
    }  
}
```

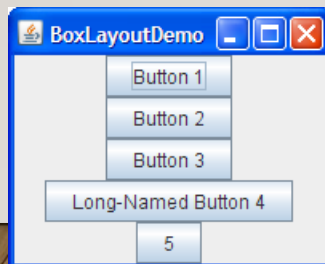
LAYOUT MANAGEMENT



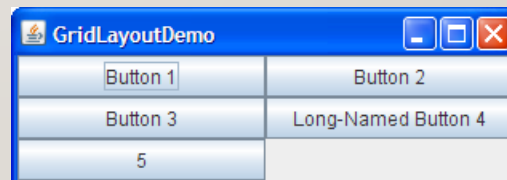
Border layout



Flow layout



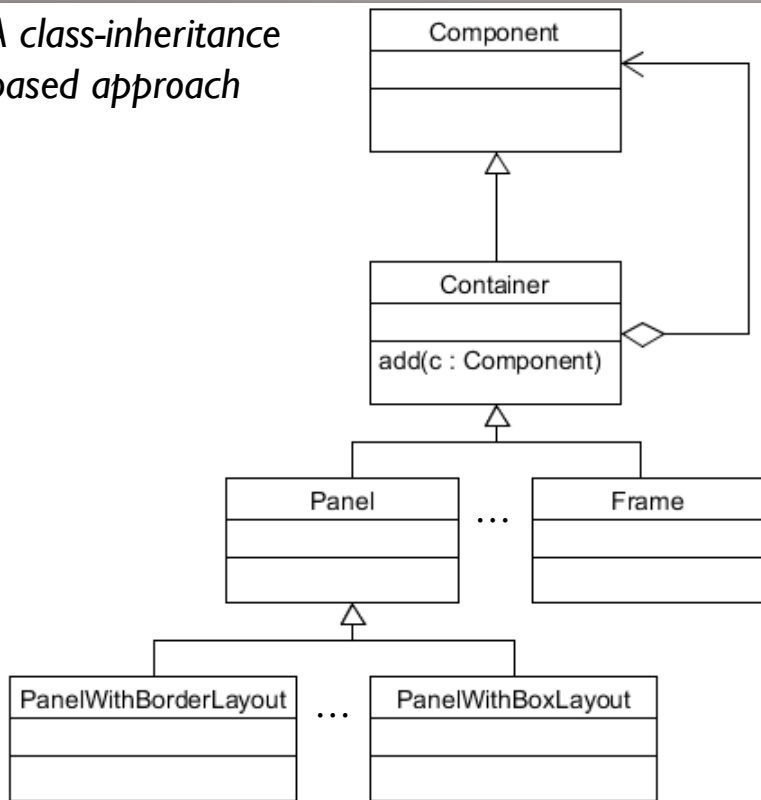
Box layout



Grid layout

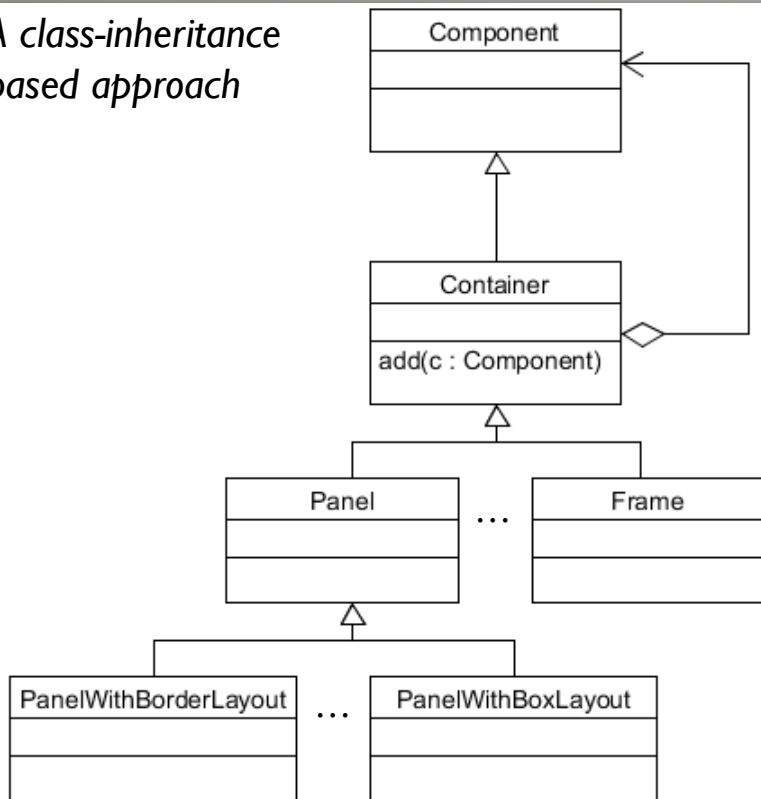
LAYOUT MANAGEMENT

*A class-inheritance
based approach*

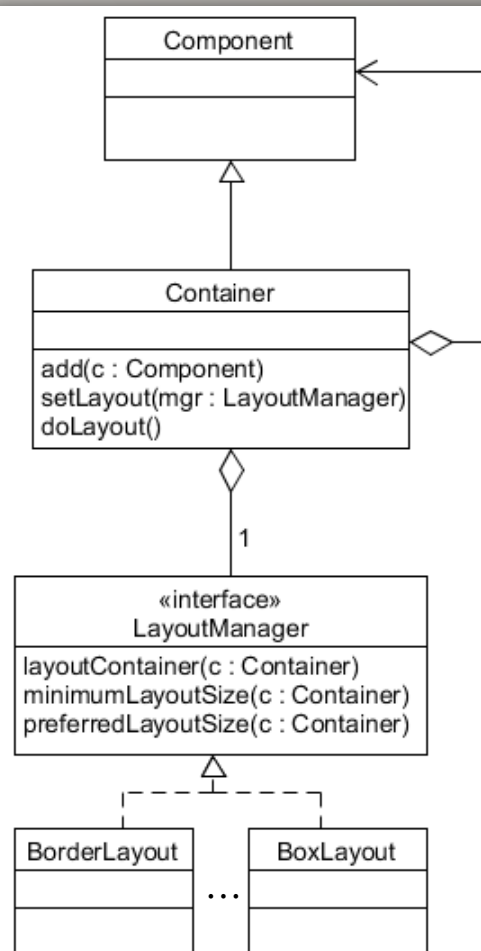


LAYOUT MANAGEMENT

A class-inheritance based approach



An aggregation- and interface-based approach



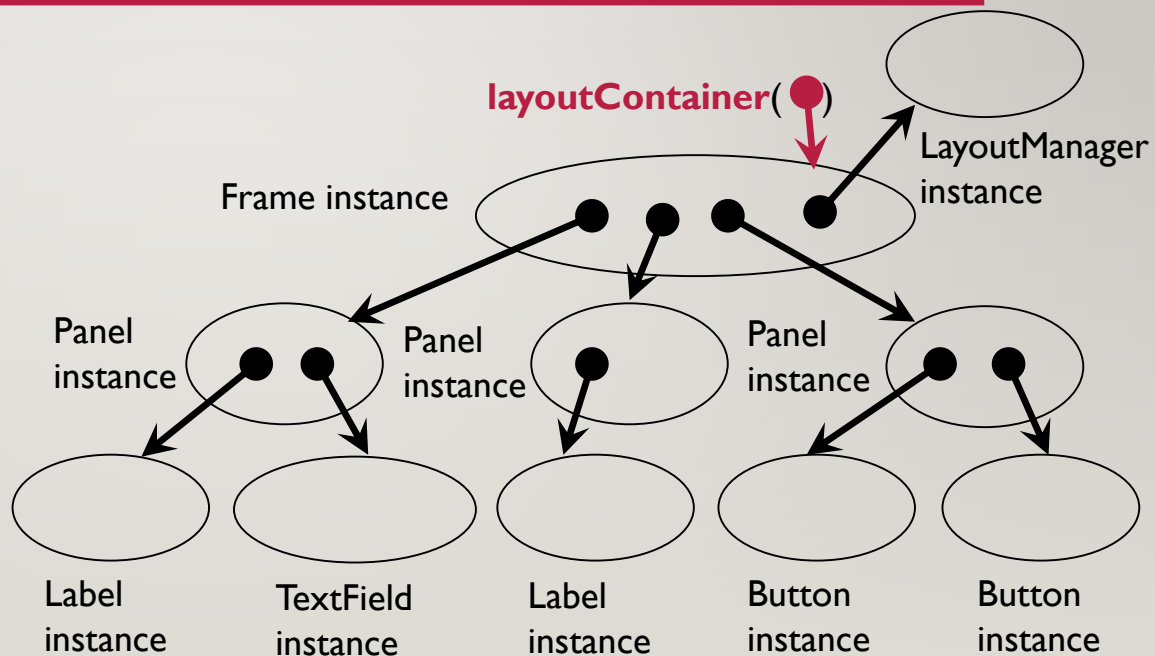
LAYOUT MANAGEMENT

```

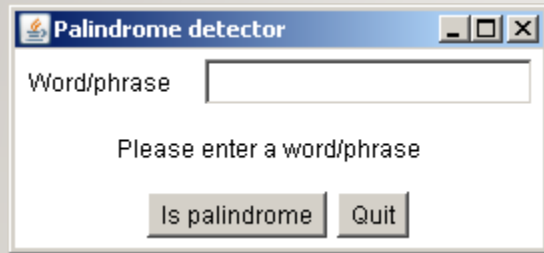
public class Container extends Component {
    ...
    private List< Component > children;
    private LayoutManager layoutManager;

    public void setLayout( LayoutManager lm ) {
        layoutManager = lm;
    }

    public void doLayout( ) {
        layoutManager.layoutContainer( this );
    }
}
    
```



EVENT HANDLING



- How do we associate behaviour with a button?
- How does a button's behaviour get executed when it's clicked?

```
import java.awt.*;
```

```
public class PalindromeGUI extends Frame {
```

```
    public PalindromeGUI() {
        super( "Palindrome detector" );
        setUp( );
```

```
        // Method inherited from class Window –
        // causes this object to be sized according
        // to its preferred size, based on its
        // children components.
        pack( );
```

```
        setVisible( true );
```

```
    }
```

```
    public static void main( String[ ] args ) {
        PalindromeGUI palindromeGUI = new
            PalindromeGUI( );
    }
```

HOW EVENT HANDLING WORKS?

Event source:

- a GUI component that generates / fires an event

Event:

- a user interaction (e.g. a click on the button)

Event listener:

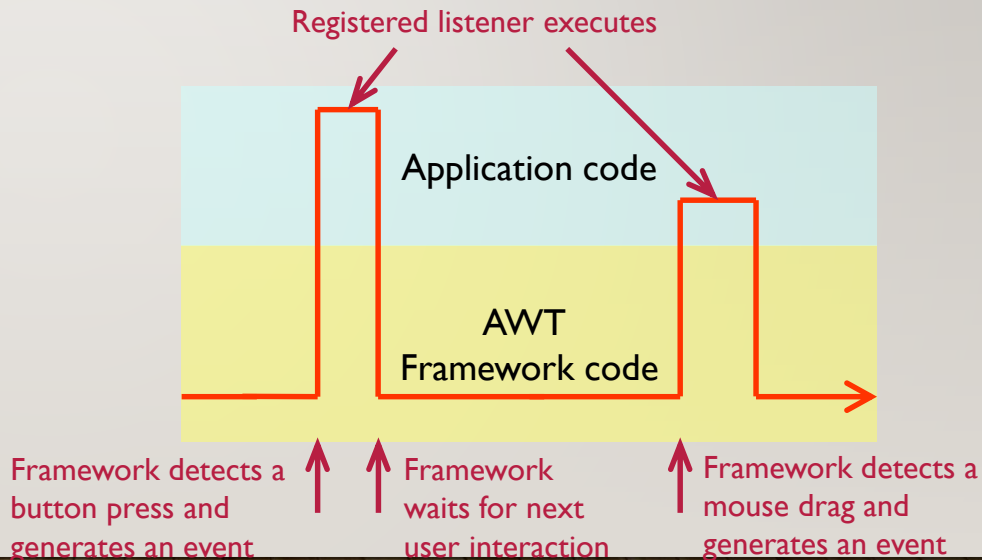
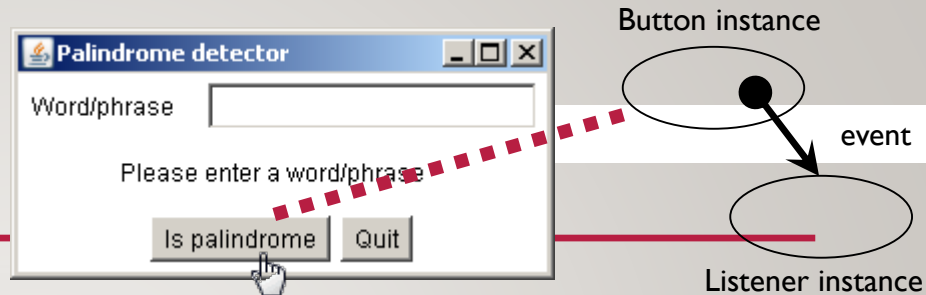
- An object that has implemented event handlers to react to an event
- **Note:** the delegation event model - event listeners must be registered with an event source in order to receive **notification**



Listener's appropriate method will be called when event occurs (e.g. when the button is clicked)

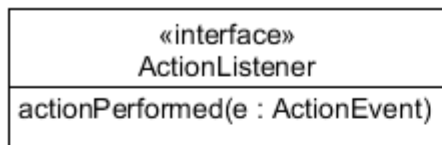
INVERSION OF CONTROL IN EVENT HANDLING

- Components generate **events** in response to user interaction
- To hook in application-specific responses to events, **listeners** are used
 - A listener is an object that is **registered** with a component; in handling user interaction, the component informs any registered listeners of events

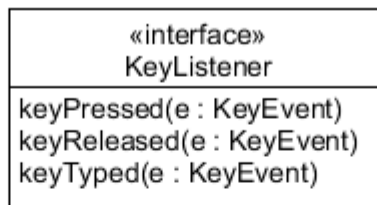


EVENT HANDLING CLASS MODEL

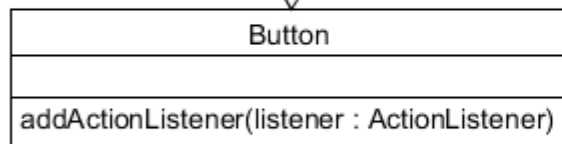
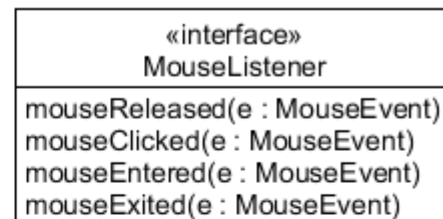
Interface for receiving action
events on a component



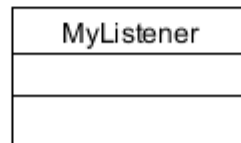
Interface for receiving key
stroke events on a
component



Interface for receiving mouse
events (e.g. release, click, enter
and exit) on a component



Framework



Application

1: addActionListener(listener)

2: actionPerformed()



Button instance

listener : MyListener

See API documentation for package `java.awt.event`
for other listener interfaces

TWO DESIGNS FOR PALINDROME GUI APP

```

public class PalindromeGUI extends Frame
    implements ActionListener {

    private TextField _text;
    private Button _isPalindrome, _quit;
    private Label _output;

    public PalindromeGUI() {
        super( "Palindrome detector" );
        ...
        _isPalindrome.addActionListener( this );
        _quit.addActionListener( this );
    }

    public void actionPerformed( ActionEvent e ) {
        if( e.getSource() == _isPalindrome ) { // isPalindrome pressed.
            String enteredText = _text.getText();
            if( enteredText.length() > 0 ) {
                if( isPalindromic( enteredText ) ) {
                    _output.setText( enteredText + " is palindromic" );
                }
                else {
                    _output.setText( enteredText + " is not palindromic" );
                }
            }
        }
        else { // quit pressed.
            System.exit( 0 );
        }
    }
}

```

```

public class PalindromeGUI extends Frame {
    private TextField _text;
    private Button _isPalindrome, _quit;
    private Label _output;

    public PalindromeGUI() {
        ...
        _isPalindrome.addActionListener( new ActionListener() {
            public void actionPerformed( ActionEvent e ) {
                String enteredText = _text.getText();

                if( enteredText.length() > 0 ) {
                    if( isPalindromic( enteredText ) ) {
                        _output.setText( enteredText + " is palindromic" );
                    }
                    else {
                        _output.setText( enteredText + " is not palindromic" );
                    }
                }
            }
        } );

        _quit.addActionListener( new ActionListener() {
            public void actionPerformed( ActionEvent e ) {
                System.exit( 0 );
            }
        } );
        ...
    }
}

```

FOUR GUI PROGRAM DESIGNS IN JAVA

(CONTD. IN THE NEXT LECTURE)

1) class **MyComponent** extends **JButton** implements **ActionListener**{

```
....  
    addActionListener(this);  
....  
}
```

2) class **MyComponent** extends **JButton**{

```
.....  
    addActionListener(new MyInnerListener());  
    private class MyInnerListener implements ActionListener{ ... }  
}
```

3) class **MyComponent** extends **JButton**{

```
.....  
    addActionListener(new ActionListener() {  
        public void actionPerformed(ActionEvent ae){....});  
    }
```

4) class **MyComponent** extends **JButton**{

```
.....  
    addActionListener(new MyIndependentListener());  
}  
class MyIndependentListener implements ActionListener{ ..}
```


STUDYING JAVA GUI LIBRARY CLASSES

- ✓ Start with Containers
- ✓ Have your own list of recognizable event sources (buttons, checkboxes, menus, etc.)
- ✓ Practice identifying various event sources from a variety of software applications you come across (your mobile apps, PC apps, etc.)
- ✓ Maintain a list of Event classes and Listener interfaces (along with their listener methods) for each event source.
- ✓ Creation of Event objects is managed by JVM, so you rarely write code to generate them.
- ✓ Event sources have many overloaded constructors and methods – need not cram!
- ✓ Remember – Example Java GUI programs will have mainly ***four designs***

REVIEW

- Learnt something about Java's key GUI frameworks:
 - Event handling is managed using the IoC principle
 - Events are detected by the framework, and through registered listeners application code is invoked
 - The developers of AWT and Swing have applied sound object-oriented principles and design patterns
 - Components (widget classes) are implemented using inheritance (*is-a*) to a generalisation/specialisation hierarchy and to enable reuse of common state and behaviour
 - Layout management is implemented using aggregation (*has-a*) to reduce the number of classes required, to avoid duplicate code and to support run-time flexibility
 - GUI component containment is the application of the Composite pattern

REFERENCES

➤ [Books]

- *Book:* P. Deitel and H. Deitel, Java How to Program, Chapter 12

• [Web Links]

- <https://docs.oracle.com/javase/8/docs/technotes/guides/awt/index.html>
- <https://www.javatpoint.com/java-awt>
- <https://docs.oracle.com/javase/tutorial/uiswing/>

➤ [See more on Canvas Lecture page]